

Адаптивный аккомпанемент в реальном времени: от классических трекеров партитуры к RL-агенту опережающего темпа

Н. И. Новицкий Н. С. Борисов А. А. Варламов

АНО Центральный Университет, мастерская MusicTech

Научный Телеграф, 2026



Структура презентации

- 1 Введение
- 2 Постановка задачи
- 3 Данные: партитура и исполнение
- 4 Классические трекары (baseline)
- 5 Почему этого недостаточно
- 6 Предлагаемый подход: RL-агент темпа
- 7 Обучение и симулятор
- 8 Итоги
- 9 Литература



Актуальность: солисты-пианисты при подготовке к концерту нуждаются в репетициях с оркестровым сопровождением, однако живой оркестр недоступен на каждой репетиции. Фиксированная фонограмма не подстраивается под темп и *rubato* солиста.

Проблема: построить виртуальный оркестр, который в реальном времени отслеживает позицию исполнителя в партитуре и упреждающе управляет темпом аккомпанемента (бюджет задержки ≤ 20 мс — порог восприятия рассинхронизации в ансамбле, Cont 2010).

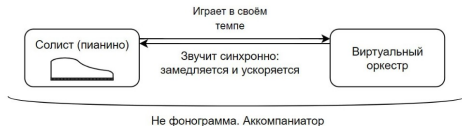
Подход: классический трекер партитуры (HMM + OLTW) + лёгкий *RL-агент*, который видит состояние трекера и управляет темпом на горизонте 200–500 мс.

Цель доклада: разобрать математическую часть baseline, обосновать формулу награды RL-агента и положение работы относительно литературы.



Проблема: аккомпаниатор, а не фонограмма

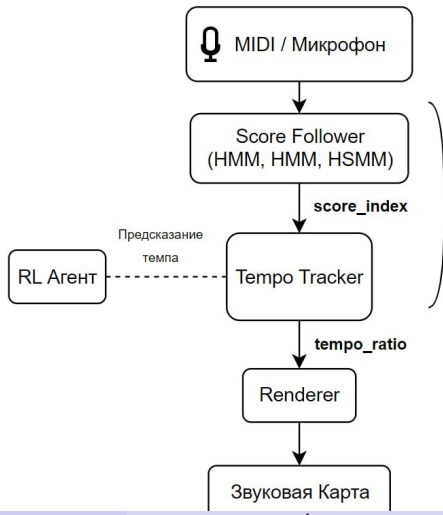
- Солисту-пианисту перед концертом нужно репетировать с **оркестром**.
- Живой оркестр на каждую репетицию — недоступно (бюджет, время, организация).
- Фиксированная фонограмма не работает: она не подстраивается под темп, артикуляцию и *rubato* солиста.
- **Цель:** виртуальный оркестр, который играет заданное сопровождение и подстраивается под солиста **в реальном времени**.



Что технически делает система

Задача формализуется как **score-following**: по поступающему MIDI/аудио-потoku и заранее известной партитуре в реальном времени определять текущую позицию исполнителя.

- **Вход:** партитура (известна) + поток нот солиста (realtime).
- **Выход:** индекс ноты $\hat{i}_t$ + темповый коэффициент τ_t .
- **Бюджет задержки:** ≤ 20 мс (порог восприятия рассинхронизации, Cont 2010).
- Core ML — классический трекер; **RL-агент опережающе** управляет темпом на 200–500 мс вперёд.

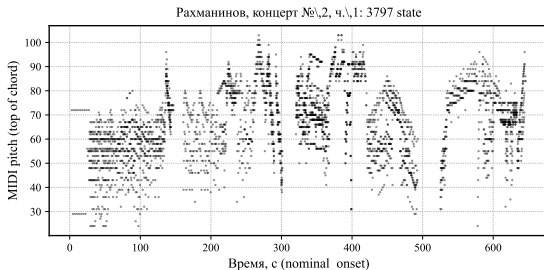


Данные: формат партитуры

Партитура хранится в JSON-формате `score.json`. Одна запись = одно скрытое состояние HMM/HSMM (может быть аккордом).

```
{  
  "index": 0,  
  "pitches": [60, 64, 67],  
  "nominal_onset": 2.999,  
  "nominal_duration": 1.499  
}
```

- `generated_dataset/` — 4 синтетические пары;
- `midi/library/rach_solo/` — Рахманинов, **3797 state**;
- для RL — импортер ASAP (Foscarin 2020): 222 пьесы, 1067 исполнений с note-level разметкой.



Реальные данные: каждая точка — состояние партитуры (верхняя нота аккорда).



Данные: исполнение в реальном времени

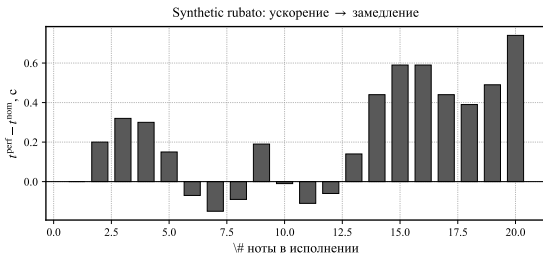
Live-поток от солиста — последовательность MIDI-событий:

```
{"pitch": 64, "timestamp": 1.234, "velocity": 88}
```

```
{"pitch": 67, "timestamp": 1.481, "velocity": 95}
```

В реальном исполнении timestamps не совпадают с nominal_onset из-за rubato, ускорений, замедлений и ошибок.

Это и есть **challenge задачи**: трекер должен оставаться синхронным, даже когда времени между нотами на лету меняется в 2–3 раза.



Типичный rubato: ускорение (отриц. дельты), потом замедление (положит.).



Dynamic Time Warping — основа OLTW (определение)

Дано: партитура $P = (p_1, p_2, \dots, p_N)$ (последовательность pitch-ов) и наблюдаемое исполнение $O = (o_1, o_2, \dots, o_T)$.

Ищем *warping path* $\pi = ((i_1, t_1), \dots, (i_L, t_L))$ с тремя свойствами:

- ❶ граничные условия: $(i_1, t_1) = (1, 1)$, $(i_L, t_L) = (N, T)$;
- ❷ монотонность: $i_{\ell+1} \geq i_\ell$ и $t_{\ell+1} \geq t_\ell$;
- ❸ шаг-1: $(i_{\ell+1} - i_\ell, t_{\ell+1} - t_\ell) \in \{(0, 1), (1, 0), (1, 1)\}$.

Стоимость пути:

$$\text{cost}(\pi) = \sum_{\ell=1}^L c(p_{i_\ell}, o_{t_\ell}), \quad c(p, o) = |p - o| \text{ (полутоны).}$$

Оптимальный путь $\pi^* = \arg \min_{\pi} \text{cost}(\pi)$.



DTW — рекуррентное соотношение (вывод)

Обозначим $D[i, t]$ — минимальная стоимость пути из $(1, 1)$ в (i, t) .

Идея вывода: последний шаг пути приходит в (i, t) из одного из трёх предков (свойство шаг-1):

- диагональ $(i-1, t-1) \rightarrow (i, t)$;
- горизонталь $(i-1, t) \rightarrow (i, t)$;
- вертикаль $(i, t-1) \rightarrow (i, t)$.

Принцип оптимальности Беллмана: путь до (i, t) оптимален $\iff$ путь до предыдущего шага оптимален. Отсюда:

$$D[i, t] = c(p_i, o_t) + \min(D[i-1, t-1], D[i-1, t], D[i, t-1])$$

с базовым случаем $D[1, 1] = c(p_1, o_1)$ и $D[\cdot, \cdot] = +\infty$ вне сетки.

Сложность: $O(NT)$ по памяти и времени.

Online-проблема: в realtime T растёт, нельзя хранить весь массив.

Решение — OLTW.



Baseline 1: OLTW (Dixon 2005)

On-Line Time Warping — инкрементальный вариант DTW.

- Локальная стоимость:

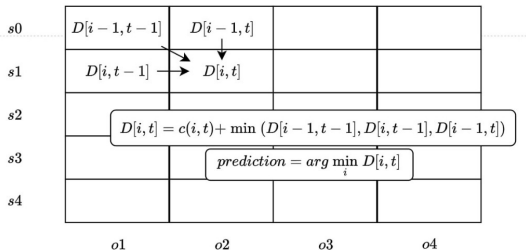
$$c(i, t) = |p_i - o_t|.$$

- Рекурсия:

$$D[i, t] =$$

$$c(i, t) + \min(D[i-1, t-1], D[i-1, t], D[i, t-1]).$$

- $\hat{i}_t = \arg \min_i D[i, t].$
- Память $O(N)$ — хранятся 2 колонки.
- RunCount* fail-safe: k событий без сдвига $\Rightarrow$ форсируем шаг.



Плюсы: простота, устойчивость к rubato, всегда выдаёт ответ. **Минусы:** нет confidence, плохо ловит структурные ошибки.



OLTW: численный пример заполнения DP-таблицы

Партитура: 5 нот гаммы С-мажор $P = (60, 62, 64, 65, 67)$.

Поток: играем первые 4 ноты ровно $O = (60, 62, 64, 65)$.

Стоимость: $c(p_i, o_t) = |p_i - o_t|$ (полутоны).

Шаг $t = 1$, $o_1 = 60$:

i	1	2	3	4	5
p_i	60	62	64	65	67
$c(p_i, o_1)$	0	2	4	5	7
$D[i, 1]$	0	2	6	11	18

(Кумулятивная сумма — это «вертикальный» шаг

$D[i, 1] = c(p_i, o_1) + D[i-1, 1]$.)

Предсказание: $\hat{i}_1 = \arg \min_i D[i, 1] = 1$ (первая нота).



OLTW: пример заполнения (продолжение)

Шаг $t = 2$, $o_2 = 62$. По рекурсии

$$D[i, t] = c(p_i, o_t) + \min(D[i-1, t-1], D[i-1, t], D[i, t-1]):$$

i	1	2	3	4	5
$c(p_i, o_2)$	2	0	2	3	5
$D[i, 2]$	2	0	2	5	10

Предсказание: $\hat{i}_2 = \arg \min_i D[i, 2] = 2$.

Шаг $t = 3$, $o_3 = 64$:

i	1	2	3	4	5
$c(p_i, o_3)$	4	2	0	1	3
$D[i, 3]$	6	2	0	1	4

$\hat{i}_3 = 3$.

Шаг $t = 4$, $o_4 = 65$: $\hat{i}_4 = 4$ (минимум в строке $i = 4$).

Итог: OLTW идеально отслеживает $\hat{i}_t = 1, 2, 3, 4$ — ровно по диагонали DP-сетки. **Память:** хранилось всего 2 колонки.



Baseline 2: HMM (Rabiner 1989)

Скрытая Марковская модель по партитуре.

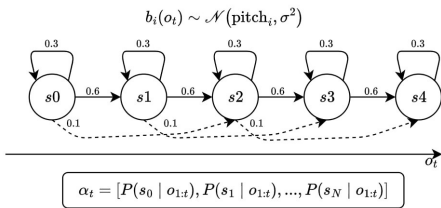
- Состояния: $i \in \{0, \dots, N-1\}$ — индексы нот.
- Эмиссия (гауссиана по pitch, σ в полутонах):

$$b_i(o) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2}\left(\frac{o-p_i}{\sigma}\right)^2\right)$$

- Переходы $\{\text{stay}=0.3, \text{advance}=0.6, \text{skip}=0.1\}$, веса зависят от elapsed в state.
- Forward-обновление:

$$\alpha_{t+1}(i) = b_i(o_{t+1}) \sum_j \alpha_t(j) a_{j,i}$$

- $\hat{i}_t = \arg \max_i \alpha_t(i)$, *confidence*
 $= \max_i \alpha_t(i)$.



HMM: вывод Forward-формулы (Rabiner 1989)

Определим $\alpha_t(i) = P(O_{1:t}, S_t = i \mid \lambda)$ — вероятность, что мы наблюдали $o_1, \dots, o_t$ и сейчас находимся в состоянии i .

Инициализация (по определению совместного):

$$\alpha_1(i) = \pi_i b_i(o_1), \quad i \in \{1, \dots, N\},$$

где $\pi_i = P(S_1 = i)$ — начальное распределение, $b_i(o) = P(o \mid S = i)$ — функция эмиссии.

Индукция $t \rightarrow t + 1$: маргинализуем по предыдущему состоянию $S_t = j$:

$$\alpha_{t+1}(i) = P(O_{1:t+1}, S_{t+1} = i) = \sum_{j=1}^N P(O_{1:t+1}, S_t = j, S_{t+1} = i).$$



HMM: вывод Forward-формулы (продолжение)

Цепной разбор по правилу произведения и марковскому свойству:

$$P(O_{1:t+1}, S_t = j, S_{t+1} = i) = \underbrace{P(O_{1:t}, S_t = j)}_{=\alpha_t(j)} \underbrace{P(S_{t+1} = i \mid S_t = j)}_{=a_{j,i}} \underbrace{P(o_{t+1} \mid S_{t+1} = i)}_{=b_i(o_{t+1})}$$

Подставляя в маргинализацию, получаем рекурсивную формулу:

$$\alpha_{t+1}(i) = b_i(o_{t+1}) \sum_{j=1}^N \alpha_t(j) a_{j,i}$$

Сложность шага: $O(N^2)$ (для каждого из N новых состояний суммируем по N предыдущим). Для нашей задачи переходы *разреженные* (только stay/advance/skip), поэтому шаг $O(N)$.



HMM: предсказание и confidence

После каждого события считаем $\alpha_t(i)$ для всех i , затем:

Предсказание позиции:

$$\hat{i}_t = \arg \max_i \alpha_t(i).$$

Уверенность (confidence): мера «остроты» распределения α_t :

$$\text{conf}_t = \frac{\alpha_t(\hat{i}_t)}{\sum_j \alpha_t(j)}.$$

Зачем это нужно для нас:

- HMM *выдаёт распределение*, а не одно число (в отличие от OLTW).
- Это распределение можно **подать на вход RL-агенту** как признак неопределённости.
- Confidence используется как переключатель в гибридной модели:
 $\text{conf}_t > \theta \Rightarrow \text{HSMM}$, иначе $\Rightarrow \text{OLTW}$.

Численная стабильность: в realtime мы нормируем α_t на сумму после каждого шага — иначе они уходят в underflow.



HMM Forward: численный пример

Партитура: 3 ноты $P = (60, 62, 64)$.

Параметры модели:

- Начальное: $\pi = (1, 0, 0)$ (стартуем с первой ноты).
- Переходы: $a_{j,j} = 0.3$ (*stay*), $a_{j,j+1} = 0.6$ (*advance*), $a_{j,j+2} = 0.1$ (*skip*).
- Эмиссия: гауссиана $b_i(o) = \frac{1}{\sigma\sqrt{2\pi}} \exp(-\frac{1}{2}(\frac{o-p_i}{\sigma})^2)$, $\sigma = 2$.

Поток: $O = (60, 62)$.

Шаг $t = 1$, $o_1 = 60$. Считаем эмиссии:

$$b_1(60) = \frac{1}{2\sqrt{2\pi}} \approx 0.199, \quad b_2(60) = 0.121, \quad b_3(60) = 0.027.$$

По формуле $\alpha_1(i) = \pi_i b_i(o_1)$ получаем $\alpha_1 = (0.199, 0, 0)$. Нормировка:
 $\alpha_1 = \mathbf{(1, 0, 0)}$.

Предсказание: $\hat{i}_1 = 1$, $\text{conf}_1 = 1.0$.



HMM Forward: численный пример (продолжение)

Шаг $t = 2$, $o_2 = 62$.

Шаг А. Подсчёт prior $\sum_j \alpha_1(j) a_{j,i}$:

$$\text{prior}_1 = 1 \cdot 0.3 = 0.3, \quad \text{prior}_2 = 1 \cdot 0.6 = 0.6, \quad \text{prior}_3 = 1 \cdot 0.1 = 0.1.$$

Шаг В. Эмиссии $b_i(62)$:

$$b_1(62) \approx 0.121, \quad b_2(62) = 0.199, \quad b_3(62) \approx 0.121.$$

Шаг С. Умножение и нормировка:

$$\tilde{\alpha}_2 = (0.0363, 0.1194, 0.0121), \quad \alpha_2 = (0.217, 0.711, 0.072).$$

Предсказание: $\hat{i}_2 = \arg \max \alpha_2 = 2$, $\text{conf}_2 = 0.711$.

Интерпретация: модель уверена что мы во второй ноте (71 %), небольшая масса остаётся на первой (мы могли ошибиться) и третьей (могли skip-нуть).
Этот вектор α_t — основной сигнал для RL-агента.

Baseline 3: HSMM (Cont 2010, эвристика)

Hidden Semi-Markov Model — расширение HMM с **явной моделью длительности**.

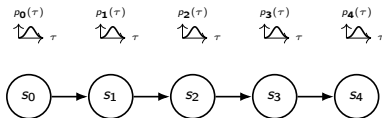
Канонический вид (Cont 2010):

$$\alpha_t(i) = \sum_{\tau=1}^D \sum_j \alpha_{t-\tau}(j) a_{j,i} p_i(\tau) \prod_{k=t-\tau+1}^t b_i(o_k)$$

где $p_i(\tau)$ — распределение длительности state i .

В нашем коде (`hsmm_follower.py`) реализована **эвристика**: переходы пересчитываются от $r = \text{elapsed} / \text{nominal_duration}$. Четыре исхода: stay / advance / skip / leap.

Добавляет к HMM: учёт длительностей $\Rightarrow$ правильное поведение при rubato и лишних нотах. **Ограничение:** наш HSMM — эвристика, не канонический Cont.



*HMM + распределение длительности $p_i(\tau)$
 $\Rightarrow$ переходы зависят от elapsed в state*



Baseline 4: Hybrid Fusion

HybridScoreFollower — HSMM + OLTW + **якорный resync**.



Правило выбора:

- $\text{conf} > \theta \Rightarrow \text{HSMM}$;
- иначе $\Rightarrow \text{OLTW}$;
- якорь нашёл сильное совпадение $\Rightarrow \text{resync}$ обоих.

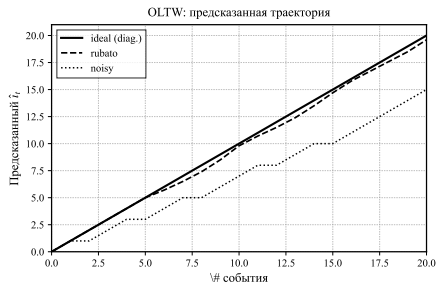
Anchor search ищет последние K событий как окно в score через template-matching по pitch + temp-fitting.

Это «классический трекер» в смысле тезисов — поверх него ставится RL-слой.

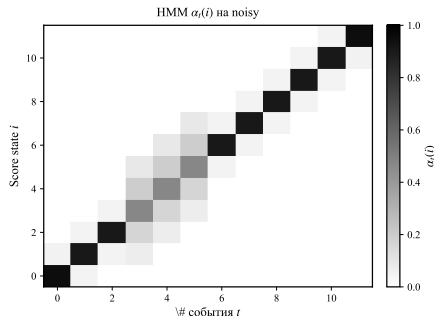


Результаты baseline на синтетике

Прогон трёх трекеров на трёх синтетических сценариях: *ideal* (гамма ровно), *rubato* (тот же pitch, expressive timing), *noisy* (лишние ноты, опечатки, пропуски).



OLTW: на *ideal/rubato* — диагональ. На *noisy* — ступеньки.



HMM α -heatmap на *noisy*: уверенность размывается.

На синтетике классические трекаеры **работают**.

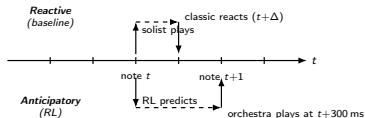
Реальный challenge — крупные пьесы (~4000 state) и индивидуальный rubato концертного исполнителя.



Почему baseline недостаточно

Три фундаментальных ограничения классических трекеров.

- 1 **Реактивность.** TempoTracker считает темп постфактум — по уже сыгранным нотам. Пока оркестр «узнаёт», что солист ускорился, тот уже играет следующую фразу. Нужно **опережающее** предсказание темпа.
- 2 **Жёсткие правила.** Эвристики (пороги confidence, deadzone темпа, anchor margin) подобраны вручную и плохо переносятся между исполнителями.
- 3 **Не используют контекст.** Trackers видят одну ноту за раз; не знают, как менялась confidence, какая динамика темпа была за 500 мс.



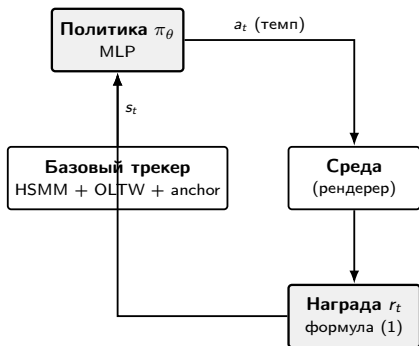
*Reactive vs anticipatory:
classic реагирует с задержкой Δ ;
RL предсказывает на 200–500 мс вперёд.*

Решение тезисов: лёгкий RL-агент видит *весь вектор состояния трекера* и предсказывает темп **вперёд по времени**.



Предлагаемая архитектура (РИС. 1 тезисов)

Идея: не заменять классический трекер, а подвесить *рядом* лёгкий RL-агент. Базовый трекер остаётся ядром. Агент читает состояние трекера и историю темпа, выдаёт темповый коэффициент $a_t \in [0.5, 2.0]$ на горизонте 200–500 мс.



Новая ниша:

- *Dorfer 2018* — RL заменяет трекер целиком.
- *Peter 2023* — RL для symbolic alignment.
- *RL-Duet, ReaLchords* — RL генерирует ноты.
- *Наш подход* — RL поверх работающего трекера, только темп, без замены классики.

Если RL-агент молчит — система деградирует до проверенного гибрида, а не ломается.



Формализация: MDP над живым трекером

RL-агент работает в среде, где базовый трекер Φ (HybridScoreFollower) — часть динамики.

Марковский процесс принятия решений $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$:

- $\mathcal{S}$ пространство состояний — то, что агент «видит» о трекере, темпе и истории (см. следующий слайд, $s_t \in \mathbb{R}^{18}$);
- $\mathcal{A}$ действие — темповый коэффициент $a_t \in [0.5, 2.0] \subset \mathbb{R}$;
- P переходная динамика — индуцируется живым HybridScoreFollower и симулятором солиста;
- R награда $r_t = R(s_t, a_t)$ — формула (1) тезисов;
- γ фактор дисконтирования, $\gamma \approx 0.99$.

Задача: найти политику $\pi_\theta(a | s)$, максимизирующую

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^H \gamma^t r_t \right].$$

Ключевое отличие от литературы: среда сохраняет *работающий*



Математика RL-слоя: состояние, действие, награда

Состояние $s_t \in \mathbb{R}^{18}$ (не зависит от длины пьесы):

$$s_t = (\hat{\alpha}_t, \tau_{t-K:t}, e_{t-K:t}, \hat{\varphi}(t)/N)$$

- $\hat{\alpha}_t$ — 7-числовая сжатка HSMM-распределения (max, entropy, argmax/ N , top-3 mass, 3 рел. индекса);
- $\tau_{t-K:t}$ — последние K оценок темпа, $e_{t-K:t}$ — эмиссионные ошибки;
- $\hat{\varphi}(t)/N$ — относительная позиция в партитуре.

Действие: $a_t \in [0.5, 2.0]$ — темповый коэффициент, отдаётся каждые 50 мс.

Награда (формула (1) тезисов):

$$r_t = \underbrace{-|t_t^{\text{render}} - t_t^{\text{perf}}|}_{\text{рассинхр.}} \underbrace{- \lambda L_{\text{align}}(t)}_{\text{ошибка трекера}} \underbrace{- \mu (a_t - a_{t-1})^2}_{\text{резкий перепад}}$$

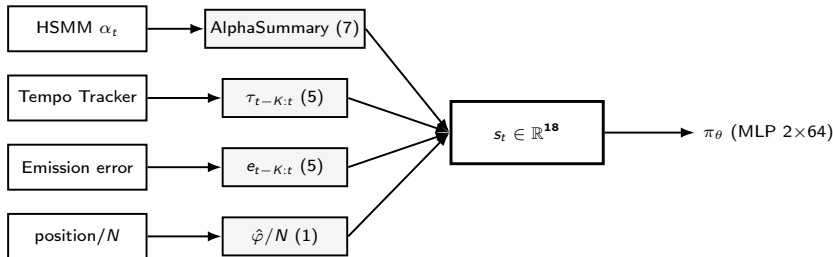
- 1-й член — основной сигнал рассинхронизации рендера и исполнителя;
- 2-й — мягкая регуляризация: не уезжать от базового трекера;
- 3-й — гладкость темпа, чтобы оркестр не «дёргался».

λ, μ подбираются на PPO по ASAP.



Энкодер состояния s_t

Все четыре источника сжимаются в один фиксированный вектор $s_t \in \mathbb{R}^{18}$ — который подаётся в MLP 2×64 .



Что важно:

- размер s_t **не зависит** от длины пьесы;
- $\hat{\alpha}_t$ — сжатка HSMM, а не сам α (иначе $\dim s_t \rightarrow N$);
- $K=5$ — баланс между контекстом и инерцией.

Что это даёт:

- одна и та же политика для пьес 200...4000 state;
- MLP инициализируется в ВС *один раз* и переносится между корпусами;
- признаки физически интерпретируемые.



Откуда взялась формула награды (1)?

Что должна минимизировать политика? Три цели, конфликтующие друг с другом:

- 1 **Точная синхронизация:** рендер оркестра в момент t_t^{render} должен совпадать с моментом исполнения солистом t_t^{perf} .
Метрика: $|t_t^{\text{render}} - t_t^{\text{perf}}|$.
- 2 **Не уезжать от классики:** агент должен опираться на работающий трекер Φ , а не фантазировать.
Метрика: $L_{\text{align}}(t)$ = дивергенция оценки темпа агента и темпа из трекера.
- 3 **Гладкость:** оркестр не должен «дёргаться».
Метрика: $(a_t - a_{t-1})^2$.

Стандартный приём: линейная комбинация трёх потерь, знак минус для максимизации:

$$r_t = -|t_t^{\text{render}} - t_t^{\text{perf}}| - \lambda L_{\text{align}}(t) - \mu (a_t - a_{t-1})^2$$

$\lambda, \mu \geq 0$ — гиперпараметры баланса.



Почему именно эти функционалы

Член 1: $|\cdot|$ а не $(\cdot)^2$.

L_1 -норма (модуль) — *медианно-робастная* к выбросам. Если исполнитель один раз сильно сбился — мы не хотим, чтобы агент панически реагировал. L_2 дал бы слишком сильную реакцию.

Член 2: L_{align} как мягкая регуляризация.

Без неё оптимизатор может найти политику, которая «забивает» на HSMM и сама придумывает темп — это уже не наша задача *дополнения* трекера. KL-дивергенция между темпом агента и темпом-оракулом из трекера штрафует уход «в фантазию».

Член 3: $(a_t - a_{t-1})^2$, не $|a_t - a_{t-1}|$.

L_2 штрафует *резкие* перепады сильнее, чем плавные. Линейный дрейф темпа (rubato) — ОК. Скачок $a_t : 1.0 \rightarrow 2.0$ за один шаг — плохо. Это *музыкально-мотивированный* выбор.

Подбор λ, μ : grid-search на ASAP, целевая метрика — RMS render-perf delay на отложенных пьесах. Типичные значения $\lambda \in [0.1, 1.0]$, $\mu \in [0.1, 5.0]$.

Обучение: двустадийная схема $BC \rightarrow PPO$

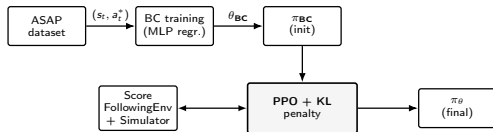
PPO с нуля плохо стартует на сложной непрерывной задаче. Используем стандартную схему $BC \rightarrow PPO + KL$ (Sequence Tutor, Jaques 2017).

Стадия А — Behavior Cloning на ASAP:

- 1 Считаем оракульный темп

$$a_t^* = \frac{\Delta \Phi_{\text{nominal}}}{\Delta \Phi_{\text{real}}} \text{ на скользящем окне.}$$

- 2 Регрессия $s_t \mapsto a_t^*$ — MLP 2×64 .
- 3 Получаем стартовую политику π_{BC} .



*ASAP даёт инициализацию,
PPO дотачивает в симуляторе.*

Стадия В — PPO + KL:

- 1 Среда ScoreFollowingEnv (Gymnasium) поверх живого HybridScoreFollower.
- 2 Симулятор солиста (rubato по Repp 1995).
- 3 Награда r_t из формулы (1).
- 4 KL-штраф $\beta D_{KL}(\pi_\theta \parallel \pi_{BC})$ против ухода в нефизические регионы.



Симулятор солиста и рендера

Для PPO нужны эпизоды — (state, action, reward) в больших количествах. На реальных ASAP-записях их мало (~ 1000 исполнений). Решение — симулятор.

Симулятор солиста:

- база — реальное ASAP-исполнение или партитура;
- параметрическое rubato (Repp 1995);
- сэмплирование ошибок (Nakamura 2015): вставка / замена / пропуск;
- диверсивность $\Rightarrow$ агент не переобучается на конкретных записях.



Замкнутый цикл: симулятор $\rightarrow$ трекер $\rightarrow$ политика $\rightarrow$ рендер $\rightarrow$ reward $\rightarrow$ обратно.

Симулятор рендера:

- детерминированное проигрывание orchestra MIDI с темпом a_t ;
- t_t^{render} — время события после temp-rescaling.

$t_t^{\text{render}} - t_t^{\text{perf}}$ доступен **без** аудио-записи оркестра — достаточно solo MIDI + orchestra MIDI.



Выводы и дальнейшая работа

Сделано:

- Систематизировали baseline (OLTW, HMM, HSMM, Hybrid) — все три классических подхода работают на синтетических сценариях.
- Реструктурировали кодовую базу под расширения, заложили типизированные DTO под RL.
- Воспроизводимые ноутбуки-демонстрации каждого компонента.

Ключевая идея:

RL-агент не заменяет классический трекер, а **дополняет** его как лёгкий слой опережающего предсказания темпа.

Это новая ниша, в литературе не покрытая.

Дальше:

- Подключить ASAP, обучить BC → PPO по описанной схеме.
- Сравнить с baseline по RMS render-perf delay и tempo jerk.
- Адаптация под конкретного исполнителя (fine-tune политики после нескольких сессий) — главное прикладное преимущество RL.

Ограничения:

- Бюджет 20 мс end-to-end — нужно подтвердить на крупных пьесах с beam-HSMM.
- Качество обучения упирается в качество ASAP-разметки и симулятора rubato.
- Микрофонный вход (требование заказчика) — отдельная подзадача, не покрывается тезисами.

Литература (классические трекары)



L. R. Rabiner.

A tutorial on hidden Markov models and selected applications in speech recognition.
Proc. IEEE 77(2):257–286, 1989.



S. Dixon.

Live Tracking of Musical Performances Using On-Line Time Warping.
Proc. DAFx, 2005.



A. Cont.

A Coupled Duration-Focused Architecture for Real-Time Music-to-Score Alignment.
IEEE Trans. PAMI 32(6):974–987, 2010.



E. Nakamura, N. Ono, S. Sagayama.

Real-time Audio-to-Score Alignment of Music Performances.
IEEE/ACM TASLP, 2015.



M. Müller.

Information Retrieval for Music and Motion.
Springer, 2007.



Литература (RL и обучение)



J. **Schulman** et al.
Proximal Policy Optimization Algorithms.
arXiv:1707.06347, 2017.



J. **Schulman** et al.
High-Dimensional Continuous Control Using Generalized Advantage Estimation.
ICLR, 2016.



R. **Sutton**, A. Barto.
Reinforcement Learning: An Introduction (2nd ed.).
MIT Press, 2018.



N. **Jaques** et al.
Sequence Tutor: Conservative Fine-Tuning of Sequence Generation Models with KL-Control.
ICML, 2017.



P. **Christiano** et al.
Deep Reinforcement Learning from Human Preferences.
NeurIPS, 2017.



Литература (RL для музыки и датасеты)



M. **Dorfer**, A. Arzt, G. Widmer.

Learning to Listen, Read, and Follow: Score Following as a Reinforcement Learning Game.
ISMIR, 2018.



S. **Peter** et al.

Online Symbolic Music Alignment with Offline Reinforcement Learning.
ISMIR, 2023.



N. **Jiang** et al.

RL-Duet: Online Music Accompaniment Generation Using Deep Reinforcement Learning.
AAAI, 2020.



Y. **Wu** et al.

Adaptive Accompaniment with ReaLchords.
NeurIPS, 2024.



F. **Foscarin** et al.

ASAP: a Dataset of Aligned Scores and Performances for Piano Transcription.
ISMIR, 2020.



C. **Hawthorne** et al.

Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset.
ICLR, 2018.



Спасибо за внимание

Вопросы?

Репозиторий: `github.com/Nizier193/music-tech`

Статья: `article/main.pdf`

Тезисы: `article/тезисы.pdf`

Код: `src/musictech-app/`

Никита Новицкий | **Никита Борисов**

Центральный Университет, мастерская MusicTech